Appendix J — Questions for your work-environment Claude (close the ownership gaps)

Source: `OFFICE_CLAUDE_QUESTIONS.md`

Office-Claude Questions — infrastructure, deployment, observability, Data Cloud

Why this file exists. The CrowdStrike round (2026-09-17) was lost on direct questions about AWS deployment and Salesforce Data Cloud. The architecture material was strong, but the infra answers weren't grounded in what you personally ran. These prompts are for a Claude instance **inside your work environment**: it can see your real repos, deploy configs, Splunk and Data Cloud setup, and this prep session can't. The goal is to turn every infra claim on the GitLab resume into something you can defend line by line, the way the ResolveFlow numbers now are.

How to use it

- Run one section per session, spread through the week (see `GITLAB_WEEK_PLAN.md`). Don't leave it all for Oct 8.
- Paste the **preamble** below first, every session.
- Take the answers into `GITLAB_KARAN_BHATIA_DEEP_DIVE.md` §14.1 and fill in the Rating column (OWN / PATTERN / THIN).
- Write down the 2–3 specifics per section you'll actually say out loud: a file name, a setting, a number, an incident timestamp.
- Respect confidentiality. In the interview you'll talk at the pattern level, with numbers rounded or relative. The point here is for *you* to know the truth underneath.
- This complements `~/Desktop/salesforce_architecture_probe.md`, which covers agent architecture, evals and permissions. This file covers infra, deployment, observability and Data Cloud.

Preamble (paste first):

> I'm preparing to explain my own work precisely in a technical interview. For every answer, cite the source: file path, config, PR, dashboard, runbook, ticket, or Slack thread. Mark each claim **VERIFIED (source)** or **INFERRED (reasoning, no source)**, and never blend them. If something isn't documented, say "not found." Don't reconstruct a plausible answer. Separately, tell me which parts I personally authored or changed (search for my commits, PRs, reviews and on-call pages) and which parts the platform/infra team owns. I need to label my role accurately as OWN / PATTERN / THIN.

A. AWS architecture for the agent services

A1. List every AWS service my agent services (the five access-management workflows, the Slack agent, and the Headless 360 / MCP integration) actually use in production: compute, API front door, queues/streams, databases, caches, secrets, networking, IAM. For each one, give the account/region model and point to where it's defined (Terraform/CDK/CloudFormation/ Helm/an internal platform manifest).

A2. For the Slack agent service specifically, what is the compute? Lambda, ECS (Fargate or EC2), EKS, or an internal Salesforce runtime on top of AWS? Show me the definition file. Give me the CPU/memory per instance/task/pod, the min/max instance counts, and the autoscaling policy (which metric, target value, cooldowns).

A3. Do any of my services use Lambda? If yes, give the runtime, memory, timeout, reserved/provisioned concurrency, trigger source, and package type (zip or container image). Give the measured cold-start latency if any dashboard has it, and the IAM execution role's policy.

A4. Is API Gateway in front of anything I own? REST API or HTTP API? Name the authorizer type, throttling/usage-plan settings, and integration timeout. If it isn't API Gateway, what is the ingress (ALB, internal gateway, Slack → which endpoint)?

A5. Why was this compute choice made over the alternatives (Lambda vs ECS vs K8s)? Find the design doc or discussion. What constraint drove it: run duration, cost, latency, compliance, or platform mandate?

A6. What does "event-driven infrastructure" on my resume concretely correspond to? Which queue/bus (SQS, SNS, Kafka/MSK, EventBridge, Salesforce Platform Events), which topics or queues my services produce and consume, and what delivery semantics apply. Is there a DLQ, what's the retry policy, and how is consumer idempotency handled? Show the consumer code path.

A7. How do my services authenticate to AWS resources and to Salesforce APIs at runtime? Name the IAM role(s), how secrets are stored (Secrets Manager / SSM / Vault / internal) and how they're rotated. Then the key question: how does a privileged action run "under the requesting user's own permissions"? Which token is used, how it's minted, its lifetime, and what happens if the user's access is revoked between request and approval.

A8. What did I personally change in the AWS/infra definitions (commits/PRs authored or reviewed by me), and what's owned entirely by another team?

B. CI/CD pipeline: stages, gates, failure and rollback

B1. Walk me through exactly how a code change to the **Slack agent service** goes from merged PR to running in production. Name every stage, every job, and every gate (tests, scans, evals, approvals). Say what system runs each one (Jenkins, GitHub Actions, an internal Salesforce CI/CD system), and what happens on failure at each stage. Cite the pipeline definition file.

B2. Same as B1 for the **access-management workflow services** and the **Headless 360 / MCP integration**. Where do the pipelines differ, and why?

B3. Is it Jenkins, GitHub Actions, or something internal? Show the Jenkinsfile / workflow YAML / pipeline config. Which agents/runners execute it? How are credentials injected?

B4. Where are container images built, how are they tagged (git SHA?), where are they scanned, and which registry stores them? Is the exact artifact that passed staging the one promoted to prod, or is it rebuilt?

B5. What's the production release strategy: rolling, canary, blue-green, or feature flags/ permission gates? Give the actual percentages, bake times, and promotion criteria. Is there an automated rollback trigger, and on what metric?

B6. How is a bad deploy rolled back in practice? Give the exact command or button, who can do it, and how long it takes. Have I ever done it? Find the instance (ticket/Slack) if so.

B7. Do the AI evaluation suites I built run in CI? Which job, on which triggers (code change, prompt change, Skill/NLD content change, model change), and do they block the release? Show the job and a recent run.

B8. (AutoRABIT era, if any artifacts are reachable) What CI/CD tooling did I build or extend there, and how was "release reliability ~30%" measured? If nothing is reachable, say so.

C. Kubernetes (or whatever we actually run on)

C1. Do my services run on Kubernetes? If yes, which cluster(s) (EKS? an internal Salesforce platform?), and which namespace(s)? Show the Deployment manifests or Helm values for each service I own.

C2. Give me the resource requests and limits for each container, and the QoS class. Has any pod been OOMKilled or CPU-throttled in the last 6 months? Show the evidence.

C3. Show the liveness, readiness and startup probes, and give terminationGracePeriodSeconds. How does the service shut down gracefully on SIGTERM, and what happens to in-flight agent work during a rolling update?

C4. Is there an HPA (or KEDA/custom autoscaler)? Which metric, what target, and what min/max replicas? Has autoscaling ever misbehaved (flapping, scaling on the wrong signal)?

C5. Show the PodDisruptionBudget, anti-affinity/topology spread, and how many replicas run per AZ.

C6. How are ConfigMaps and Secrets used? Are secrets mounted from an external secrets manager? How do config changes roll out?

C7. Show the Service/Ingress/gateway path from the outside world (Slack, internal callers) to the pod.

C8. What did I personally author or tune in these manifests (git blame / PRs)? Which parts are platform-team templates I inherited?

D. Splunk and observability wiring, plus one real incident reconstructed

D1. Which Splunk indexes and sourcetypes hold logs for my services? Show a representative structured log line. Which fields are always present (correlation/trace id, tenant/org id, user id, workflow, node/stage, tool, status, latency)?

D2. Is there distributed tracing (OpenTelemetry, an internal tracer)? Which attributes propagate across service hops? Does trace context survive async hops (queue messages) and the human-approval pause, or does the trace break there?

D3. Show the actual SPL queries I've run or saved for debugging these services. Give me the 3 most useful ones verbatim, with what each answers.

D4. Which dashboards and alerts exist for my services? For each alert: the metric, threshold, severity, and who gets paged. Which ones did I create?

D5. Are there SLOs/SLIs defined (availability, latency, success rate)? Give the targets and how burn rate is alerted.

D6. Find a real production incident on a service I own or was on-call for. Reconstruct it with timestamps: alert/symptom → the queries run → the hypotheses → the root cause layer (retrieval, planning, tool/API, auth/token, cache, infra) → mitigation → fix → follow-up action items, and whether a regression/eval case was added. Cite the ticket/postmortem/Slack thread. I need to tell this from memory: what did I personally do at each step?

D7. Give me a second, smaller incident, preferably one where the symptom showed up in one service but the cause was in a dependency (an auth/token, cache, or rate-limit issue).

E. Data Cloud / Data 360: specifics for my own systems

E1. Which of my workflows or agents actually read from Data Cloud / Data 360, and through which path (QueryDataSource action, retriever, SFAP AI Gateway, UKD, the NLD search index)? Which don't touch it at all?

E2. For the data my agents retrieve, describe the ingestion path: source → connector (batch or streaming) → DLO → DMO → Chunk DMO → embedding → vector/keyword index. Which of these stages are configured for my team's content, and by whom? Did I configure any of them?

E3. What are the actual freshness numbers for the content my agents use: ingestion cadence, re-embedding/refresh cadence, and index rebuild triggers? Is there a dashboard? Does the "CRM ingestion ~10–15

minutes / identity resolution 1–24 hours" guidance apply to my data, or does my content flow through a different path (e.g., the mDLO metadata-extraction pipeline: New → Processing → Syncing → Data_Copied → Ingestion_Requested → Ingestion_Complete)?

E4. How is tenant isolation enforced for my retrieval calls? Confirm the orgJWT / TokenExchange handler path, the token's scope and lifetime, and where it's minted. Is ACL metadata applied before retrieval (pre-filter) or after?

E5. What retrieval parameters does my integration use: top-k, score thresholds, metadata filters, hybrid/re-ranking settings? Where are they configured, and were they ever tuned using eval data?

E6. Is the NLD/Skill content served from Core's classpath rather than round-tripping to C360? Does that optimization apply to my workflows?

E7. Have there been incidents or bugs caused by Data Cloud staleness, ingestion failure, or retrieval misconfiguration affecting my agents? Reconstruct one.

E8. In one paragraph, what can I honestly say I *did* with Data Cloud (configured, debugged, consumed, evaluated) as opposed to what I only know about?

F. "Throughput up ~40%" and "on-call rotation": methodology and evidence

F1. Find the source of the "throughput up ~40% with stronger fault isolation" claim: a dashboard, a design doc, a launch/retro note, or a performance review. What was the exact metric and unit (completed executions/hour? requests/sec? jobs/min)?

F2. What were the baseline and comparison windows, and the load conditions (peak vs average, same traffic mix)? Was resource budget held constant (same replica count / instance size)?

F3. What exactly changed to produce it? Bulkheading per workflow, sync → async queue consumers, right-sizing requests/limits, autoscaling on a different signal, a downstream fix? Point to the PRs, and say which were mine.

F4. How was "stronger fault isolation" demonstrated: an incident where one downstream's failure no longer affected other workflows, a chaos/load test, or reasoning only?

F5. If the 40% can't be traced to a measurement, what defensible statement can I make instead (e.g., a different measured number, or the mechanism without the number)?

F6. For the on-call rotation: which services, what rotation cadence, which paging tool, and how many pages did I take in the last 6–12 months? Pull the paging history if it's accessible.

F7. Which runbooks exist for my services? Did I write or update any? Show them.

F8. Pick my most significant on-call page and reconstruct the first 15 minutes: acknowledge, assess blast radius, mitigate, communicate. What did I do, in what order?

F9. Were there postmortems I authored or contributed to? Summarize the action items and whether they shipped.

After each session: write these down

- My **role** per topic: OWN / PATTERN / THIN (into DD §14.1).
- **Three specifics** I'll say out loud: a config value, a number with its method, an incident with timestamps.
- **One honest boundary sentence** per topic: "The platform team owned X; I owned Y."
- Anything that contradicts a resume claim. Decide **before Oct 9** how you'll phrase it.
